iZprime Framework: Pseudocode Documentation

https://github.com/Zprime137/iZprime

January 21, 2026

Introduction

The *iZprime* framework is a C toolkit for prime generation tasks and development, featuring implementations of various prime sieve algorithms. This document provides pseudocode representations of the key algorithms implemented in the iZprime framework, along with explanations and complexity analyses.

The pseudocode presented here abstracts away implementation details to focus on the core logic of each algorithm.

Contents

1 Basic Data Structures **2**
- 1.1 Integer Array (`INT_ARRAY`) . . . 2
- 1.2 Bitmap (`BITMAP`) . . . 2

2 Prime Sieve Algorithms **2**
- 2.1 Sieve of Eratosthenes (SoE) . . . 3
- 2.2 Segmented Sieve of Eratosthenes (SSoE) . . . 3
- 2.3 Sieve of Euler (SoEu) . . . 4
- 2.4 Sieve of Sundaram (SoS) . . . 5
- 2.5 Sieve of Atkin (SoA) . . . 6
- 2.6 Sieve-iZ (SiZ) . . . 8
- 2.7 Segmented Sieve-iZm (SiZm) . . . 9

3 Summary **12**

A Algebraic Study on the iZ Function **13**

1 Basic Data Structures

Prime sieve algorithms in this document utilize two basic data structures: an integer array (`INT_ARRAY`) for storing the list of primes, and a bitmap (`BITMAP`) for efficiently representing candidate numbers and their primality states.

1.1 Integer Array (`INT_ARRAY`)

To store and manage the list of prime numbers generated by the sieve algorithms, we define an integer array structure called `INT_ARRAY`, which holds a dynamic array of integers and their count. A simple definition of the `INT_ARRAY` structure in C could be as follows:

```
typedef struct
{
    size_t capacity;       ///< Total capacity (max count of elements) of the array
    int count;             ///< Number of elements currently stored
    int *array;            ///< Pointer to the dynamically allocated int array
} INT_ARRAY;
```

In this document, we use the following operations on `INT_ARRAY`:

- `int_array_init()`: Initializes and returns an `INT_ARRAY` object with an array with dynamic *capacity* and *count* initialized to 0.
- `int_array_push(`*int_array*, *value*`)`: Appends the integer *value* to the end of *int_array*.

1.2 Bitmap (`BITMAP`)

To efficiently represent a range of numbers in sieve algorithms, we define a structure called `BITMAP`, which is a compact array of bits, where the *bit-index* represents a candidate number and the *bit-state* (0 or 1) encodes its primality state. A simple definition of the `BITMAP` structure in C could be as follows:

```
typedef struct
{
    size_t size;        ///< Total number of bits in the bitmap
    uint8_t *data;      ///< A dynamically allocated array of bytes ((size + 7) / 8)
} BITMAP;
```

In this document, we use the following operations on `BITMAP`:

- `bitmap_init(`*size*, *initial_value*`)`: Initializes and returns a bitmap of given *size* with all bits are set to *initial_value*.
- `bitmap_set_all(`*bitmap*`)`: Sets all bits in the *bitmap* to 1.
- `bitmap_get_bit(`*bitmap*, *index*`)`: Returns the bit-state at *index* in the *bitmap*.
- `bitmap_flip_bit(`*bitmap*, *index*`)`: Flips the bit-state at *index* in the *bitmap*.
- `bitmap_clear_steps(`*bitmap*, *step*, *start*, *limit*`)`: Clears bits in the *bitmap* starting from *start* and incrementing by *step* until *limit* is reached.
- `bitmap_clone(`*bitmap*`)`: Creates and returns a copy of the given *bitmap*.

2 Prime Sieve Algorithms

This section presents pseudocode for various prime sieve algorithms, along with their complexity analyses. All algorithms return an ordered list of primes up to a given limit N.

2.1 Sieve of Eratosthenes (SoE)

The Sieve of Eratosthenes algorithm (SoE), attributed to the ancient Greek mathematician Eratosthenes, is a fundamental algorithm for finding all prime numbers up to a specified limit N.

It works by iteratively marking the multiples of primes up to $\sqrt{N}$. The unmarked numbers that remain are the prime numbers.

Algorithm 1 Sieve of Eratosthenes (SoE)

```
procedure SoE(int : n) → INT_ARRAY
                                                    ▷ 1. Initialization:
    INT_ARRAY: primes ← INT_ARRAY_INIT()            ▷ initialize primes array
    INT_ARRAY_PUSH(primes, 2)                       ▷ append 2 to primes, to focus on odd candidates
    BITMAP: sieve ← BITMAP_INIT(n + 1, 1)           ▷ create sieve bitmap with all bits set to 1
                                                    ▷ 2. Sieve Process: mark composites and collect primes
    n_sqrt ← ⌈√n⌉                                   ▷ compute root limit
    for i ← 3 to n with step 2 do                   ▷ iterate over odd candidates
        if BITMAP_GET_BIT(sieve, i) then            ▷ if i is prime
            INT_ARRAY_PUSH(primes, i)               ▷ append i to primes
            if i < n_sqrt then                      ▷ if i is root prime
                BITMAP_CLEAR_STEPS(sieve, 2i, i^2, n)   ▷ mark its odd multiples starting from i^2
            end if
        end if
    end for
                                                    ▷ 3. Return primes
    return primes
end procedure
```

Complexity Analysis

Space Complexity: Since SoE maintains one *solid* bitmap of size $N+1$, its auxiliary space requirements is a **linear** $O(N)$, which is acceptable for small to moderate N, but becomes prohibitive for large N.

Speedups: This implementation of SoE applies trivial optimizations to improve efficiency. For instance, it skips even numbers entirely since they are not prime (except for 2), and it starts marking odd composites from p^2 for each prime $3 \leq p \leq \sqrt{N}$. The bit-work after applying these optimizations is estimated as:

$$O\left(\frac{N}{2}\sum_{p=3}^{\sqrt{N}}\frac{1}{p}\right).$$

Time Complexity: Since the cost-per-prime for marking composites is $1/p$, the time complexity is $O(N \log\log N)$, but with an improved constant factor ($N/2$ instead of the naive N).

2.2 Segmented Sieve of Eratosthenes (SSoE)

The Segmented Sieve of Eratosthenes (SSoE) is a memory-efficient variant of the classic SoE algorithm. Instead of allocating a single large bitmap of size N, SSoE divides the range into smaller segments of size Δ (often $\sqrt{N}$) and processes each segment independently. The algorithm first generates all root primes up to $\sqrt{N}$, then uses these primes to sieve each subsequent segment.

This approach maintains the same time complexity as SoE while significantly improving cache locality and allowing the algorithm to scale to much larger limits.

Algorithm 2 Segmented Sieve of Eratosthenes (SSoE)

```
procedure SSoE(int : n) → INT_ARRAY
                                                          ▷ 1. Initialization:
    segment_size ← ⌈√n⌉                                   ▷ compute segment size
    BITMAP: segment ← BITMAP_INIT(segment_size + 1, 1)    ▷ create segment bitmap
    INT_ARRAY: primes ← SoE(segment_size)                 ▷ initialize primes array with root primes up to √n
                                                          ▷ 2. Process subsequent segments
    low ← segment_size + 1                                ▷ initialize segment bounds
    high ← low + segment_size − 1
    while low ≤ n do                                      ▷ iterate over segments
        BITMAP_SET_ALL(segment)                           ▷ reset segment bitmap
        for p ≥ 3 ∈ primes do                             ▷ sieve current segment using root primes
            if p² > high then break
            end if                                        ▷ no need to process larger primes
            start ← max(p², ⌈low/p⌉ × p)                  ▷ find first multiple of p in current segment
            start ← start if start mod 2 ≠ 0 else start + p   ▷ ensure start is odd
            BITMAP_CLEAR_STEPS(segment, 2p, start − low, high − low)   ▷ mark multiples
        end for
        for i ← low to high with step 2 do                ▷ collect primes from segment
            if BITMAP_GET_BIT(segment, i − low) then      ▷ if unmarked, it's prime
                INT_ARRAY_PUSH(primes, i)
            end if
        end for
        low ← low + segment_size                          ▷ move to next segment
        high ← min(high + segment_size, n)
    end while
                                                          ▷ 3. Return primes
    return primes
end procedure
```

Complexity Analysis

Space Complexity: SSoE requires only $O(\sqrt{N})$ auxiliary space for the segment bitmap, plus $O(\pi(\sqrt{N}))$ for storing root primes. This is a significant improvement over SoE's $O(N)$ space requirement, making it suitable for finding primes up to very large limits.

Speedups: Like SoE, SSoE skips even numbers and starts marking from p^2. The key advantage is improved cache locality, resulting in dramatically faster memory access patterns compared to the monolithic SoE bitmap. For each segment, we only mark multiples using primes up to $\sqrt{high}$.

Time Complexity: The time complexity remains $O(N \log \log N)$, but with improved constant factors due to better cache utilization.

2.3 Sieve of Euler (SoEu)

The Sieve of Euler, also known as the Linear Sieve, is an efficient prime-finding algorithm that marks each composite number exactly once. Unlike the Sieve of Eratosthenes, which may mark a composite multiple times (e.g., 15 is marked when sieving with 3 and when sieving with 5), the Sieve of Euler ensures that each composite n is marked only by its smallest prime factor.

This property makes the algorithm truly linear in the number of operations, though in practice it may be slower than optimized versions of SoE due to less favorable memory access patterns.

Algorithm 3 Sieve of Euler (SoEu)

```
procedure SoEu(int : n) → INT_ARRAY
                                                    ▷ 1. Initialization:
    INT_ARRAY: primes ← INT_ARRAY_INIT()            ▷ initialize primes array
    INT_ARRAY_PUSH(primes, 2)                       ▷ add 2 to primes
    BITMAP: sieve ← BITMAP_INIT(n + 1, 1)           ▷ create sieve bitmap
                                                    ▷ 2. Sieve Process: collect primes and mark composites
    for i ← 3 to n with step 2 do                   ▷ iterate over odd candidates
        if BITMAP_GET_BIT(sieve, i) then            ▷ if i is prime
            INT_ARRAY_PUSH(primes, i)               ▷ add i to primes
        end if
        for p > 2 ∈ primes do                       ▷ mark multiples using collected primes
            if p × i > n then break                 ▷ stop if product exceeds n
            end if
            BITMAP_CLEAR_BIT(sieve, p × i)          ▷ mark p × i as composite
            if i mod p = 0 then break               ▷ crucial: stop at smallest prime factor
            end if
        end for
    end for
                                                    ▷ 3. Return primes
    return primes
end procedure
```

Complexity Analysis

Space Complexity: The algorithm requires $O(N)$ space for the sieve bitmap and $O(\pi(N))$ for storing the primes list.

Speedups: The key optimization is the inner loop termination condition: when $i \bmod p = 0$, we know p is the smallest prime factor of i, so any composite of the form $i \times q$ for primes $q > p$ will be marked later by its smallest prime factor. This ensures each composite is marked exactly once. The algorithm skips even numbers by starting at 3 and incrementing by 2. The bit-work is estimated as:

$$O\left(\frac{N}{2}\right),$$

for marking each odd composites only once.

Time Complexity: The algorithm achieves true $O(N)$ time complexity since each number is visited at most once during the marking phase. However, the constant factors are typically higher than optimized SoE implementations due to the inner loop over all collected primes and less predictable memory access patterns.

2.4 Sieve of Sundaram (SoS)

The Sieve of Sundaram is an algorithm for finding all prime numbers up to a specified limit N. It works by eliminating numbers of the form $i + j + 2ij$ from a list of odd numbers, where $1 \leq i \leq j$ and $i + j + 2ij \leq k = (N + 1)/2$. After eliminating these numbers, the remaining numbers i in the range $[1, k]$ correspond to primes via the formula $2i + 1$.

This algorithm is conceptually elegant and naturally produces only odd primes, requiring special handling only for the prime 2. It operates on a reduced space of size $N/2$, similar to skipping even numbers in other sieves.

Algorithm 4 Sieve of Sundaram (SoS)

```
procedure SoS(int : n) → INT_ARRAY
                                                        ▷ 1. Initialization:
    INT_ARRAY: primes ← INT_ARRAY_INIT()                ▷ initialize primes array
    INT_ARRAY_PUSH(primes, 2)                           ▷ add 2 (special case)
    k ← ⌊(n+1)/2⌋ + 1                                   ▷ compute the size of the odd set up to n
    BITMAP: sieve ← BITMAP_INIT(k, 1)                   ▷ create sieve bitmap
                                                        ▷ 2. Sieve Process: mark non-primes and collect primes
    n_sqrt ← ⌈√n⌉ + 1                                   ▷ root limit
    for i ← 1 to k do                                   ▷ iterate through Sundaram space
        if BITMAP_GET_BIT(sieve, i) then                ▷ if i is unmarked
            p ← 2i + 1                                  ▷ compute prime from index
            INT_ARRAY_PUSH(primes, p)                   ▷ add prime
            if p < n_sqrt then                          ▷ if p is root prime
                x_p ← p × i + i                         ▷ first composite index: corresponds to p^2
                BITMAP_CLEAR_STEPS(sieve, p, x_p, k)    ▷ mark composites additively with step p
            end if
        end if
    end for
                                                        ▷ 3. Return primes
    return primes
end procedure
```

Complexity Analysis

Space Complexity: The algorithm requires **linear** $O(N/2)$ space for the sieve bitmap, which is half the size of the classical SoE.

Speedups: The Sieve of Sundaram originally marks composites using a multiplicative approach, targeting odd composites of the form $2(i + j + 2ij) + 1$, yielding a time complexity of $O(N \log N)$. However, in this implementation, we optimize the marking process by using the Xp-Identity, allowing us to mark composites additively with a step size of p. This modification reduces the bit-work to:

$$O\left(\frac{N}{2}\sum_{p=3}^{\sqrt{N}}\frac{1}{p}\right).$$

Time Complexity: The time complexity for this implementation is $O(N \log\log N)$ with constant factors similar to basic SoE.

2.5 Sieve of Atkin (SoA)

The Sieve of Atkin is a modern algorithm for finding all prime numbers up to a specified limit N. It uses a different approach from the classical Sieve of Eratosthenes, based on number theory results involving quadratic forms. The algorithm first marks candidates by testing specific modular arithmetic conditions, then eliminates multiples of prime squares.

Algorithm 5 Sieve of Atkin (SoA)

```
procedure SoA(int : n) → INT_ARRAY
                                                                    ▷ 1. Initialization:
    INT_ARRAY: primes ← INT_ARRAY_INIT()                            ▷ initialize primes array
    INT_ARRAY_PUSH(primes, 2)                                       ▷ add 2 and 3 as special cases
    INT_ARRAY_PUSH(primes, 3)
    BITMAP: sieve ← BITMAP_INIT(n + 1, 0)                           ▷ create sieve bitmap, initially all 0
                                  ▷ 2. Preprocessing: Mark potential primes using Atkin conditions
    for x ← 1 while 4x^2 < n do                                     ▷ Condition 1: 4x^2 + y^2 ≡ 1 (mod 12) or 5 (mod 12)
        for y ← 1 while 4x^2 + y^2 ≤ n do
            b ← 4x^2 + y^2
            if b mod 12 = 1 or b mod 12 = 5 then
                BITMAP_FLIP_BIT(sieve, b)                           ▷ flip bit
            end if
        end for
    end for
    for x ← 1 while 3x^2 < n do                                     ▷ Condition 2: 3x^2 + y^2 ≡ 7 (mod 12)
        for y ← 1 while 3x^2 + y^2 ≤ n do
            b ← 3x^2 + y^2
            if b mod 12 = 7 then
                BITMAP_FLIP_BIT(sieve, b)
            end if
        end for
    end for
    for x ← 1 while 2x^2 < n do                                     ▷ Condition 3: 3x^2 − y^2 ≡ 11 (mod 12) and x > y
        for y ← x − 1 down to 1 do
            b ← 3x^2 − y^2
            if b > n then break
            end if
            if b mod 12 = 11 then
                BITMAP_FLIP_BIT(sieve, b)
            end if
        end for
    end for
                                  ▷ 3. Sieve: Eliminate composites by removing multiples of squares
    n_sqrt ← ⌈√n⌉
    for p ← 5 to n_sqrt with step 2 do                              ▷ iterate over odd numbers ≥ 5
        if BITMAP_GET_BIT(sieve, p) then                            ▷ if p survived, it's prime
            BITMAP_CLEAR_STEPS(sieve, 2p^2, p^2, n)                 ▷ clear odd multiples of p^2
        end if
    end for
                                                                    ▷ 4. Collect remaining primes
    for p ← 5 to n with step 2 do
        if BITMAP_GET_BIT(sieve, p) then
            INT_ARRAY_PUSH(primes, p)
        end if
    end for
                                                                    ▷ 5. Return primes
    return primes
end procedure
```

Complexity Analysis

Space Complexity: The algorithm requires $O(N)$ space for the sieve bitmap.

Speedups: The Sieve of Atkin's main advantage comes from its use of quadratic forms to identify primes more directly. By testing specific modular conditions, many numbers are identified as prime (or composite) without the repeated marking of the Eratosthenes approach. The bit-flipping mechanism in the first phase accounts for the fact that some numbers satisfy multiple quadratic forms. The final elimination phase removes only odd multiples of squares of primes, resulting in an incredible cost-per-prime of $N/(2p^2)$. The total bit-work can be approximated as:

$$O\left(N\left(\frac{1}{4}+\frac{1}{6}+\frac{1}{12}\right)+\frac{N}{2}\sum_{p=5}^{\sqrt{N}}\frac{1}{p^2}\right),$$

which simplifies to $O(N)$.

Time Complexity: The theoretical time complexity is a linear $O(N)$, which is asymptotically better than SoE's $O(N \log\log N)$. However, in practice, for typical ranges up to 10^{12}, highly optimized SoE or segmented variants often outperform SoA due to better cache utilization and simpler operations.

2.6 Sieve-iZ (SiZ)

The Sieve-iZ (SiZ) algorithm uses a basic wheel optimization. Focusing only on the candidate space of $\pm 1 \pmod 6$, denoted as $i\mathbb{Z}$, this approach reduces the space requirements from N to $N/3$, and the cost-per-prime for marking composites to $N/(3p)$.

Starting with the definitions and notations that inform the SiZ algorithms:

Definition 2.1 (The $i\mathbb{Z}$ set)**.** The union of two disjoint partitions modulo 6:

$$i\mathbb{Z} \;=\; \{z \in \mathbb{N} : z \equiv \pm 1 \pmod 6\} \;=\; i\mathbb{Z}^- \dot\cup\, i\mathbb{Z}^+, \quad i\mathbb{Z}^- = \{6x-1 : x \in \mathbb{N}\}, \quad i\mathbb{Z}^+ = \{6x+1 : x \in \mathbb{N}\},$$

with cardinality $|i\mathbb{Z}| = \frac{1}{3}|\mathbb{N}|$.

To map between indices x and $6x \pm 1$ values, we use the iZ function

Definition 2.2 (The iZ function)**.**

$$\boxed{iZ(x \in \mathbb{N},\ i \in \{\pm 1\}) = 6x + i}.$$

Here, x is the *index* in a partition and i indicates which partition (-1 for $i\mathbb{Z}^-$ and $+1$ for $i\mathbb{Z}^+$).

X-Arrays. In sieving $i\mathbb{Z}$ up to a limit N, we maintain two separate bitmaps, one per partition, each of length $x_n = \lceil \frac{N}{6} \rceil$ bits:

- `X5[`x`]` encodes the candidate $6x-1$ (the $i\mathbb{Z}^-$ lane),
- `X7[`x`]` encodes the candidate $6x+1$ (the $i\mathbb{Z}^+$ lane).

The Xp-Identity, Composite Identity within $i\mathbb{Z}$. The Xp-Identity is a modular form that provides an index-based, additive marking scheme for marking composites of a given prime within the $i\mathbb{Z}$ space. Derived from Lemma A.1, it allows us to compute the composite indices of a prime p in both partitions using simple arithmetic operations on indices, avoiding multiplicative calculations.

Definition 2.3 (Xp-Identity)**.** Let $p = iZ(x_p, i_p) \in \mathbb{P}$. Then the composite indices of p in the X-Arrays are

$$\begin{aligned}
&\texttt{X5:} \quad \left\{\, p\,x_q - i_p\,x_p \;:\; x_q \geq x_p \in \mathbb{N} \,\right\} \text{ maps to the composites of } p \text{ in } i\mathbb{Z}^-,\\
&\texttt{X7:} \quad \left\{\, p\,x_q + i_p\,x_p \;:\; x_q \geq x_p \in \mathbb{N} \,\right\} \text{ maps to the composites of } p \text{ in } i\mathbb{Z}^+.
\end{aligned}$$

Here, the lower bound $x_q \geq x_p$ (for the first hit) is to avoid duplicate marking of composites whose smaller cofactor would have been handled earlier.

Algorithm 6 Sieve-iZ (SiZ)

```
procedure SiZ(int : n) → INT_ARRAY
                                                        ▷ 1. Initialization
    INT_ARRAY: primes ← INT_ARRAY_INIT()                ▷ initialize primes array and add 2 and 3
    INT_ARRAY_PUSH(primes, 2)
    INT_ARRAY_PUSH(primes, 3)
    x_n ← ⌊(n+1)/6⌋ + 1                                 ▷ iZ upper bound
    BITMAP: x5 ← BITMAP_INIT(x_n, 1)                    ▷ create bitmap x5 for representing iZ− candidates
    BITMAP: x7 ← BITMAP_INIT(x_n, 1)                    ▷ create bitmap x7 for representing iZ+ candidates
                                                        ▷ 2. Sieve: mark composites and collect primes
    n_sqrt ← ⌈√n⌉                                       ▷ root limit
    for x ← 1 to x_n do                                 ▷ iterate over iZ candidates
        if BITMAP_GET_BIT(x5, x) then                   ▷ if iZ− prime
            p ← iZ(x, −1)                               ▷ compute p
            INT_ARRAY_PUSH(primes, p)                   ▷ append p to primes
            if p < n_sqrt then                          ▷ mark multiples of p in both x5 and x7 using the Xp identity
                BITMAP_CLEAR_STEPS(x5, p, p · x + x, x_n)
                BITMAP_CLEAR_STEPS(x7, p, p · x − x, x_n)
            end if
        end if
        if BITMAP_GET_BIT(x7, x) then                   ▷ if iZ+ prime, similar process
            p ← iZ(x, 1)
            INT_ARRAY_PUSH(primes, p)
            if p < n_sqrt then
                BITMAP_CLEAR_STEPS(x5, p, p · x − x, x_n)
                BITMAP_CLEAR_STEPS(x7, p, p · x + x, x_n)
            end if
        end if
    end for
                                                        ▷ 3. Return primes
    return primes
end procedure
```

Complexity Analysis

Space Complexity: For maintaining two bitmaps of size $(N/6)$ each, it requires **linear** $O(N/3)$ auxiliary space.

Time Complexity: Additive marking within the $i\mathbb{Z}$ space $(6x \pm 1)$ yields bit-cost

$$O\left(\frac{N}{3}\sum_{p=5}^{\sqrt{N}}\frac{1}{p}\right),$$

which is $O(N \log\log N)$, but with improved constant-factors over classic SoE.

2.7 Segmented Sieve-iZm (SiZm)

The Segmented Sieve-iZm (SiZm) algorithm extends the SiZ approach by incorporating segmentation and wheel factorization up to 19, improving both space and time efficiency.

Key Ideas

In segmenting the $i\mathbb{Z}$ space ($i\mathbb{Z}^-$ and $i\mathbb{Z}^+$), SiZm processes each candidate stream in fixed-size segments of length VX_k (the product of the first k primes greater than 3 and up to 19). This strategy allows the algorithm to handle large limits N without requiring a massive contiguous bitmap, while also leveraging wheel factorization to pre-sieve the base primes (primes that divide the wheel VX_k).

To keep the algorithm compact and clean, we abstract reusable components into separate procedures:

Compute optimal segment size *VX*. The `iZm_compute_limited_vx` procedure calculates the maximum segment size VX that does not exceed $\lfloor (n+1)/6 \rfloor$, by multiplying the first k primes greater than 3.

```
INT_ARRAY: s_primes = [5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]   ▷ global list of primes greater than 3
procedure IZM_COMPUTE_LIMITED_VX(n, k)
    x_n ← ⌊(n+1)/6⌋
    vx ← 1
    i ← 0
    for p ∈ s_primes while i < k do                    ▷ iterate over first k primes greater than 3
        if vx × p > x_n then break                     ▷ stop if it exceeds x_n
        end if
        vx ← vx × p
        i ← i + 1
    end for
    return vx
end procedure
```

Construct pre-sieved base bitmaps. The `iZm_construct_vx_base` procedure sieves the base bitmaps for a segment of size VX using all primes that divide VX. These bitmaps are then reused for initializing each segment's active bitmaps, skipping the marking of those small primes during segment sieving.

```
procedure IZM_CONSTRUCT_VX_BASE(vx, base_x5, base_x7)
    for p ∈ s_primes such that p | vx do                ▷ iterate over primes dividing vx
        ip ← 1 if p mod 6 == 1 else −1
        x_p ← ⌊(p+1)/6⌋
        bitmap ← base_x5 if ip == −1 else base_x7       ▷ select which bitmap that p belong to
        BITMAP_CLEAR_BIT(bitmap, x_p)                   ▷ mark p itself
                                   ▷ mark composites of p in base bitmaps using the Xp-Identity
        BITMAP_CLEAR_STEPS(base_x5, p, p · x_p − ip · x_p, vx)
        BITMAP_CLEAR_STEPS(base_x7, p, p · x_p + ip · x_p, vx)
    end for
end procedure
```

The *Xp* solver. The `iZm_solve_for_xp` procedure computes the first composite index of a prime p in a given segment y, based on the Xp-Identity.

```
procedure SOLVE_FOR_X(m_id, p, vx, y)
    ip ← 1 if p mod 6 == 1 else −1
    xp ← ⌊(p+1)/6⌋
    if y == 0 then
        return p · xp + m_id · ip · xp                  ▷ first composite index for p in row 0
    end if
    xp ← xp if m_id == ip else p − xp                   ▷ normalize xp
    x ← p − ((vx · y − xp) mod p)
    return x                                            ▷ first composite index for p in row y
end procedure
```

Algorithm 7 Segmented Sieve-iZm (SiZm)

```
procedure SiZm(int : n) → INT_ARRAY
                                                                    ▷ 1. Initialization
    vx ← iZm_compute_limited_vx(n, 6)                               ▷ compute segment size up to vx6
    ▷ Initialize primes array with SiZ(iZ(vx, 1)) to sieve first segment and collect enough root primes
    INT_ARRAY: primes ← SiZ(iZ(vx, 1))
                                                                    ▷ Initialize base bitmaps, pre-sieved by primes dividing vx
    BITMAP: base_x5 ← bitmap_init(vx, 1)                            ▷ create base bitmap for iZm5
    BITMAP: base_x7 ← bitmap_init(vx, 1)                            ▷ create base bitmap for iZm7
    iZm_construct_vx_base(vx, base_x5, base_x7)                     ▷ sieve base bitmaps by primes dividing vx
                                                                    ▷ 2. Sieve remaining vx segments for y in [1, y_limit]
    x_n ← ⌊(n+1)/6⌋ + 1                                             ▷ iZ upper bound
    y_limit ← ⌊x_n/vx⌋                                              ▷ number of full segments
    for y ← 1 to y_limit (inclusive) do
                                                                    ▷ a. Initialize active bitmaps from the pre-sieved base bitmaps
        BITMAP: x5 ← bitmap_clone(base_x5)                          ▷ a copy bitmap for active sieve in iZm5
        BITMAP: x7 ← bitmap_clone(base_x7)                          ▷ a copy bitmap for active sieve in iZm7
                                                                    ▷ b. Mark composites of root prime in current segment
        x_limit ← vx if (y < y_limit) else x_n mod vx               ▷ full segment or partial last segment
        yvx ← y × vx                                                ▷ compute base value for current segment
        root_limit ← √(6 · (yvx + x_limit)) + 1                     ▷ compute root limit for current segment
        for p ≥ 5 ∈ primes do                                       ▷ iterate over primes greater than 3
            if p | vx then continue                                 ▷ skip pre-sieved primes
            end if
            if p > root_limit then break                            ▷ stop the marking loop beyond local root limit
            end if
                                                                    ▷ mark composites of p in current segment using iZm xp solver
            bitmap_clear_steps(x5, p, iZm_solve_for_xp(−1, p, vx, y), x_limit)
            bitmap_clear_steps(x7, p, iZm_solve_for_xp(1, p, vx, y), x_limit)
        end for
                                                                    ▷ c. Collect primes from current segment
        for x ← 1 to x_limit (inclusive) do
            if bitmap_get_bit(x5, x) then                           ▷ i.e. iZ- prime
                int_array_push(primes, iZ(yvx + x, −1))
            end if
            if bitmap_get_bit(x7, x) then                           ▷ i.e. iZ+ prime
                int_array_push(primes, iZ(yvx + x, 1))
            end if
        end for
    end for
                                                                    ▷ 3. Return primes
    return primes
end procedure
```

Complexity Analysis

Space Complexity: Instead of arbitrary segment size $\sqrt{N}$ in traditional segmented sieve, SiZm decouples the segment size from N, and instead uses a maximum wheel

$$VX_6 = 5 \times 7 \times 11 \times 13 \times 17 \times 19 = 1,616,615$$

as a fixed segment size. Thus, other than the $O(\pi(\sqrt{N}))$ for root primes, for each segment, it requires two bitmaps of maximum size VX_6 (1.6 million bits $\approx$ 200 KB) each, independent of N. This design choice ensures cache-resident operations.

Speedups: Building on the SiZ optimizations, SiZm further reduces the bit-work by pre-sieving small primes that divide VX_6 (i.e., 5, 7, 11, 13, 17, and 19) in the wheel structure. This significantly reduces the bit-work required for marking composites, as these small primes contribute the most to the total marking cost. The bit-work after skipping these primes is estimated as:

$$O\left(\frac{N}{3}\sum_{p=23}^{\sqrt{N}}\frac{1}{p}\right)+O(1),$$

where the $O(1)$ factor is a negligible cost of wheel construction.

Time Complexity: $O(N\log\log N)$, but with very improved constant factors.

3 Summary

Algorithm	**Space Complexity**	**Time Complexity**	**Bit-Work**
SoE	$O(N)$	$O(N\log\log N)$	$O\left(N\sum_{p=2}^{\sqrt{N}}\frac{1}{p}\right)$
SSoE	$O(\pi(\sqrt{N}))+O(\sqrt{N})$	$O(N\log\log N)$	$O\left(N\sum_{p=2}^{\sqrt{N}}\frac{1}{p}\right)$
SoEu	$O(N)$	$O(N)$	$O\left(\frac{N}{2}\right)$
SoS	$O(N/2)$	$O(N\log\log N)$	$O\left(\frac{N}{2}\sum_{p=3}^{\sqrt{N}}\frac{1}{p}\right)$
SoA	$O(N)$	$O(N)$	$O\left(\frac{N}{2}+\frac{N}{2}\sum_{p=5}^{\sqrt{N}}\frac{1}{p^2}\right)$
SiZ	$O(N/3)$	$O(N\log\log N)$	$O\left(\frac{N}{3}\sum_{p=5}^{\sqrt{N}}\frac{1}{p}\right)$
SiZm	$O(\pi(\sqrt{N}))+O(1)$	$O(N\log\log N)$	$O\left(\frac{N}{3}\sum_{p=23}^{\sqrt{N}}\frac{1}{p}\right)+O(1)$

Table 1: Summary of Prime Sieve Algorithms

A Algebraic Study on the iZ Function

Lemma A.1. *Let $m = 6$ be a constant modulus. Let $\mathbb{I} = \{\pm 1\}$ be the set of residues coprime to m, closed under multiplication. Let $iZ(x \in \mathbb{N}, i \in \mathbb{I}) = mx + i$.*

Let $p = iZ(x_p, i_p)$ and $q = iZ(x_q, i_q)$, then their product $z = pq = iZ(x_z, i_z) \in i\mathbb{Z}$, where

$$x_z = mx_p x_q + i_p x_q + i_q x_p, \quad i_z = (i_p i_q) \in \mathbb{I}.$$

Consequently, this equivalence holds for each prime factor p that divides z:

$$p \mid z \quad \Longleftrightarrow \quad x_z \equiv (\frac{i_z}{i_p}) x_p \pmod{p}.$$

Proof. Expanding the product $z = pq$ gives:

$$z = pq = (mx_p + i_p)(mx_q + i_q) = m^2 x_p x_q + m i_p x_q + m i_q x_p + i_p i_q.$$

Grouping terms,

$$z = m(mx_p x_q + i_p x_q + i_q x_p) + i_p i_q = mx_z + i_z.$$

This shows that z can be expressed in the form $iZ(x_z, i_z)$, where

- The *product residue* $i_z \in \mathbb{I}$ is the product of the factors residues i_p and i_q, yielding:

$$i_z = \begin{cases} 1 & \text{if } i_p = i_q, \\ -1 & \text{if } i_p \neq i_q. \end{cases}$$

- The *product index* x_z can be expressed in terms of the factors p and q as:

$$x_z = mx_p x_q + i_p x_q + i_q x_p = px_q + i_q x_p = qx_p + i_p x_q,$$

  such that x_z satisfies

$$x_z \equiv \begin{cases} i_q x_p \pmod{p}, \\ i_p x_q \pmod{q}. \end{cases}$$

  From this equivalence, we derived the Xp-Identity, which pinpoints the composite indices of a given prime p in the X-arrays using additive steps of p.

□

References

[1] Wikipedia contributors. Sieve of Eratosthenes. In *Wikipedia, The Free Encyclopedia*. Retrieved January 1, 2026, from `https://en.wikipedia.org/wiki/Sieve_of_Eratosthenes`

[2] Wikipedia contributors. Segmented sieve. In *Wikipedia, The Free Encyclopedia*. Retrieved January 1, 2026, from `https://en.wikipedia.org/wiki/Sieve_of_Eratosthenes#Segmented_sieve`

[3] Wikipedia contributors. Euler's sieve. In *Wikipedia, The Free Encyclopedia*. Retrieved January 1, 2026, from `https://en.wikipedia.org/wiki/Sieve_of_Eratosthenes#Euler's_sieve`

[4] Wikipedia contributors. Sieve of Sundaram. In *Wikipedia, The Free Encyclopedia*. Retrieved January 1, 2026, from `https://en.wikipedia.org/wiki/Sieve_of_Sundaram`

[5] Wikipedia contributors. Sieve of Atkin. In *Wikipedia, The Free Encyclopedia*. Retrieved January 1, 2026, from `https://en.wikipedia.org/wiki/Sieve_of_Atkin`